
Test-Time Compute Allocation: Search vs. Verify

Author Name

Affiliation

The Concept Given a fixed inference budget, a model can spend it two ways: generate many candidate answers and take the most frequent one (search), or generate fewer answers and check each one with a judge (a verifier). Under a weak judge and a moderate budget (the setting of this course), search usually wins. The best split depends on your model, your judge, and your budget.

Leveling and Prerequisite Knowledge Level 1 (no prerequisites): watch the five-minute video and read this page; you get the core trade-off in plain terms. Level 2 (systematic): work through the notebooks and slides; it is aimed at graduate students and early-career researchers in LLM reasoning and is adaptable for advanced undergraduates. Before starting, you should answer: (1) Why does an LLM reload all weights for every token? (2) If each candidate is right 30% of the time, what is the chance that majority voting over three independent candidates is right? (3) What do MATH and GSM8K measure, and how do their difficulties differ? Answers are built into Notebook 00.

Learning Objectives Level 1: after the video, explain the search–verification trade-off in plain language, including the two conditions (weak judge, moderate budget); rubric: a 30-second retelling. Level 2: (1) estimate failure rates with the simplified model and predict the candidate count N you need; rubric: a calculation. (2) run the notebooks, plot your own budget–accuracy curve with at least two data points, and read the six-model curve family; rubric: a plot plus a written reading. (3) given a new task and budget, reason out the best split; rubric: an analysis report. (4) advanced: judge whether a paper’s baselines are fair; rubric: a critical report.

Core Principle Search has a mathematical guarantee: with independent candidates and accuracy p above $1/2$, the failure probability of majority voting falls exponentially in N . Verification has no such guarantee; it only helps if the judge can tell good candidates from bad ones. Here the judge is the model grading its own answers; in our measurements that judge is weak, because self-scores barely separate correct from incorrect candidates. Verification value therefore depends on judge quality.

Worked Example Task: MATH 500 subset (100 problems), budget 50 calls per problem, Qwen3-4B, three seeds. Model spectrum (six models, measured p from 0.37 to 0.73): Gemma-3-1B, Phi-4-Mini, Qwen3-0.6B, Gemma-3-4B, Qwen3-1.7B, Qwen3-4B. Results are in Table 1.

Table 1: Measured accuracy on the 100-problem MATH 500 subset (Qwen3-4B, three seeds, budget 50 calls per problem).

Strategy	Split	Accuracy [95% CI]	Reference
Search	50 candidates, majority vote	0.853 [0.81, 0.89]	[2] (Majority)
Search + light verification	25 candidates $\times$ 1 check	0.717 [0.67, 0.77]	–
Verification	5 candidates $\times$ 9 checks	0.753 [0.70, 0.80]	[2] (PRM weighted)

These conclusions are limited to this configuration: a weak self-judge, a moderate budget, and the MATH 500 subset. Other tasks, including GSM8K, need separate testing (Experiment 3 checks whether the conclusion transfers). All measurements use the 100-problem subset; the literature column is directional only: the reference rows use different-scale, unquantized, earlier-generation models (the process reward model, PRM, row uses a strong trained reward model) and measure budget in FLOPs. Our judge is a weak self-judge counting inference calls on 2025 open models in GGUF Q8_0. Our weak-judge setting falls within the weak-verifier regime examined in *When To Solve, When To Verify* [5].

Decision Tree How to choose: First, ask how often your model is right. At or below 50%, voting has no guarantee; run a small probe. Between 50% and 60%, the exponential bound is weak; probe first. Above 60%, voting has a guarantee. Second, ask how strong your judge is. Weak judge: search wins. Strong judge: search first, then light verification. Budget shapes the split: below roughly ten calls, always search; at a hundred or more with a strong judge, search first and verify (about half and half); without a strong judge, keep searching. This tree is a hypothesis to test, not a settled rule; the notebooks let you check it on your own model.

Three Progressive Experiments Experiment 1: budget efficiency. Plot search and verification accuracy against calls used, and read the six-model p-spectrum. Experiment 2: the best mixed split. With the budget identity $N \times (1 + M) = 50$, find the optimal cell on a heatmap. Experiment 3: transfer. Predict the best split on GSM8K from the model, then test it. Advanced Experiment 4: critique the baselines in a test-time-compute paper.

Linked Papers The concept taught here is stated first and stands on its own; the papers below apply test-time compute allocation. Sample, Scrutinize and Scale [1] shows that sampling-based search scales with verification and beats voting at very large budgets, a reverse example at the far end. Scaling LLM Test-Time Compute Optimally [2] is the budget-efficiency comparison behind our literature column: compute-optimal allocation beats fixed best-of-N by up to $4\times$, and with a strong judge (e.g., a trained reward model), verification-guided search wins at low to moderate budgets. At very high budgets, best-of-N is more reliable. Let Me Think! [3] contrasts parallel and serial expansion: many short chains versus one long chain. Provable Scaling Laws [4] gives pairwise-comparison aggregation a guarantee even below $p = 1/2$, a contrast to majority voting.

Materials Roadmap Start with the video (five minutes, intuition), then this page, then the notebooks in order. Each experiment runs in about one to two hours on a small local model; a cached version runs instantly; a large GPU model takes three to four hours. Pick a model from the six-model spectrum that fits your device. The slides add the math; the architecture diagram is a one-page visual summary. The learning path in Figure 1 routes you through these materials. Sample noise on runs of a few dozen problems or fewer can reach $\pm 20\%$; treat such runs as pilots that place your model into broad bands, not as verdicts.

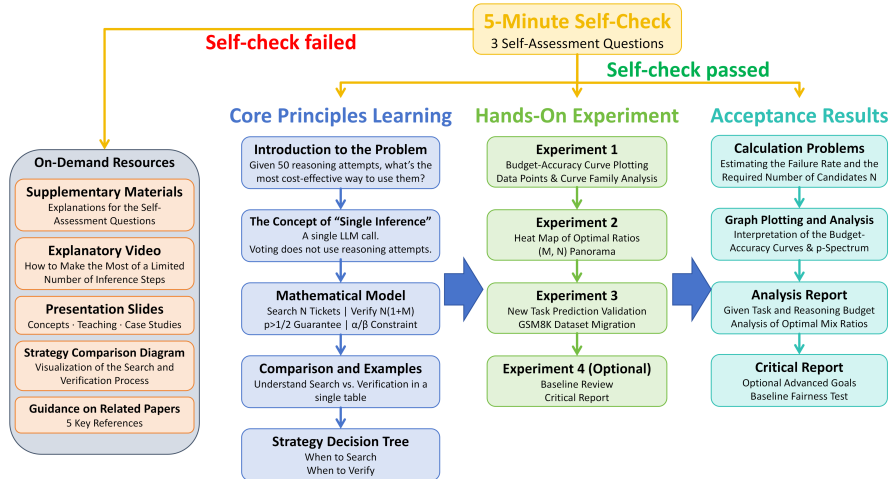


Figure 1: Learning path: a five-minute self-check routes learners to on-demand resources or to the three-path sequence (core principles, hands-on experiments, acceptance results).

Teaching Materials Summary Video (1080p, about five minutes), four notebooks with precomputed caches, lecture slides, an architecture diagram, and this repository's scripts and data, all available at <https://github.com/Solitus0726/search-vs-verify-course>. All materials are original and were created for the NeurIPS 2026 Education Track.

References

- [1] Zhao, E., Awasthi, P., & Gollapudi, S. (2025). [Sample, Scrutinize and Scale: Effective Inference-Time Search by Scaling Verification](#). ICML 2025. arXiv:2502.01839.
- [2] Snell, C., Lee, J., Xu, K., & Kumar, A. (2025). [Scaling LLM Test-Time Compute Optimally Can be More Effective than Scaling Parameters for Reasoning](#). ICLR 2025. arXiv:2408.03314.
- [3] Mirtaheiri, P., Edelman, E., Jelassi, S., Malach, E., & Boix-Adserà, E. (2025). [Let Me Think! A Long Chain-of-Thought Can Be Worth Exponentially Many Short Ones](#). NeurIPS 2025. arXiv:2505.21825.
- [4] Chen, Y., Pan, X., Li, Y., Ding, B., & Zhou, J. (2025). [Provable Scaling Laws for the Test-Time Compute of Large Language Models](#). NeurIPS 2025. arXiv:2411.19477.
- [5] Singhi, N., Bansal, H., Hosseini, A., Grover, A., Chang, K.-W., Rohrbach, M., & Rohrbach, A. (2025). [When To Solve, When To Verify](#). COLM 2025. arXiv:2504.01005.